Dr. D. Y. Patil Educational Federation's

Dr. D. Y. Patil Technical Campus

Varale Campus, Pune

Department of Electronics & Computer Engineering

Vision of the Institute

To become a leading institution in engineering education, fostering a culture of innovation, excellence and ethical leadership for a digitally transformed world.

Mission of the Department

1. To provide high-quality engineering education, fostering critical thinking, problem-solving and creativity.

2. To promote a culture of innovation, research and development

3. To cultivate strong partnership with industry leaders, ensuring our curriculum aligns with industry demands.

4. To engage with the community, providing technological solutions to local and global challenges.

5. To instil strong sense of ethical responsibility among students of Engineering.

6. To promote a culture of lifelong learning, encouraging students to adapt to the ever-changing technology landscape.

Dr. D. Y. Patil Educational Federation's
Dr. D. Y. Patil Technical Campus
Varale Campus, Pune
Department of Electronics & Computer Engineering

Course Objectives

The objective of this course is to provide students with

1. To develop hands-on skills in interfacing peripheral devices such as LED, LCD, DAC, and motors with 8051 and RL78 microcontrollers using embedded C programming.
2. To understand and apply timer, interrupt, and multiplexing techniques for controlling displays, sensors, and actuators in embedded system applications.
3. To develop and implement microcontroller-based circuits for real-time signal generation, motor control, and sensor-based monitoring using standard driver ICs and modules.

   To familiarize students with industry-relevant development tools and hardware platforms including 8051 and RL78 for building practical embedded applications.

Course Outcomes

CO1: Interface and program LED banks, seven-segment displays, LCD, and RGB LEDs with 8051 and RL78 microcontrollers using timer and interrupt operations.

CO2: Develop embedded C programs for DAC waveform generation, stepper motor control, and DC motor speed/direction control using 8051 microcontrollers

CO3: Interface IR obstacle sensor, LDR, and LM35 temperature sensor with RL78 microcontroller for real-time detection and monitoring applications

CO4: Apply ADC operation, echo pulse timing, and digital output control techniques using RL78 microcontroller for sensor-based embedded system development.

Dr. D. Y. Patil Educational Federation's
Dr. D. Y. Patil Technical Campus
Varale Campus, Pune
Department of Electronics & Computer Engineering

PCC-305-ECE: PCC-305-ECE: Advanced Java Programming & Microcontrollers and Applications Lab

Group B: Microcontrollers and Applications Lab	
1.	To interface an LED bank with the 8051-microcontroller using timer and interrupt operation for sequential LED blinking and timing control applications.
2	To interface a Seven Segment Display with the 8051 microcontrollers for displaying numeric digits and counter values using multiplexing technique.
3	To interface a stepper motor with the 8051 microcontrollers for controlling rotational direction, step angle, and speed operation
4	To interface a 16×2 LCD display module with the 8051 microcontroller and display alphanumeric characters, messages, and system status information.
5	To interface an LED with the RL78 microcontroller and write a program for LED blinking for understanding digital output port operations and delay generation.
6	To interface a 16×2 LCD display module with the RL78 microcontroller and display alphanumeric characters, messages, and system status information.

Dr. D. Y. Patil Educational Federation's
Dr. D. Y. Patil Technical Campus
Varale Campus, Pune
Department of Electronics & Computer Engineering

Experiment No. 1

Aim:

To interface an LED bank with the 8051-microcontroller using timer and interrupt operation for sequential LED blinking and timing control applications.

Objective:

1. To be familiar with software delay calculations using 8051 microcontroller
2. To find functions of P0,P1,P2,P3 of 8051 Microcontroller

Apparatus: PC, Keil simulator

Theory: Examining Figure 2.1, note that of the 40 pins, a total of 32 pins are set aside for the four ports P0, P1, P2, and P3 where each port takes 8 pins. The rest of the pins are designated as V_{cc}, GND, XTAL1, XTAL2, RST, EA, PSEN, and ALE**.**

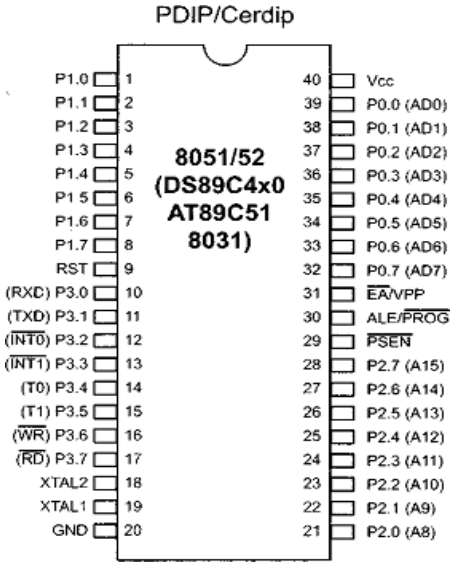


Figure 2.1 Pin Diagram of 8051 Microcontroller

Of these pins, six (V_{cc}, GND, XTAL1, XTAL2, RST, and EA) are used by all members of the 8051 and 8031 families. In other words, they must be connected in order for the system to work, regardless of whether the microcontroller is of the 8051 or 8031 family. The other two pins, PSEN and ALE, are used mainly in 8031-based systems. We first describe the function of each pin. Ports are discussed separately.

XTAL1 and XTAL2

The 8051 has an on-chip oscillator but requires an external clock to run it. Most often a quartz crystal oscillator is connected to inputs XTAL1 (pin 19) and XTAL2 (pin 18). The quartz crystal oscillator connected to XTAL1 and XTAL2 also needs two capacitors of 30 pF value. One side of each capacitor is connected to the ground

It must be noted that there are various speeds of the 8051 family. Speed refers to the maximum oscillator frequency connected to XTAL. For example, a 12-MHz chip must be connected to a crystal with 12 MHz frequency or less. Likewise, a 20-MHz microcontroller requires a crystal frequency of no more than 20 MHz. When the 8051 is connected to a crystal oscillator and is powered up, we can observe the frequency on the XTAL2 pin using the oscilloscope.

If you decide to use a frequency source other than a crystal oscillator, such as a TTL oscillator, it will be connected to XTAL1; XTAL2 is left unconnected

RST- Pin 9 is the RESET pin. It is an input and is active high (normally low). Upon applying a high pulse to this pin, the microcontroller will reset and terminate all activities This is often referred to as a *power-on reset.* Activating a power-on reset will cause all values in the registers to be lost. It will set program counter to all Os. Figures 2-3 (a) and (b) show two ways of connecting the RST pin to the power-on reset circuitry.

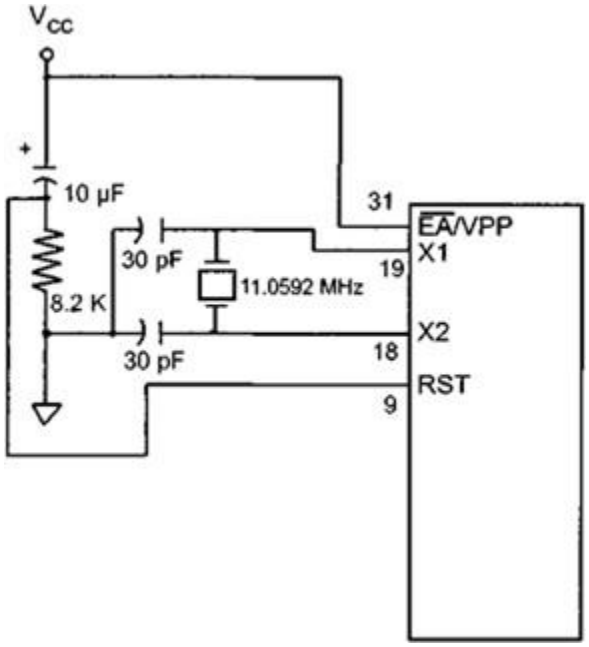


Figure 2-3(a) RST pin to the power-on reset circuitry.

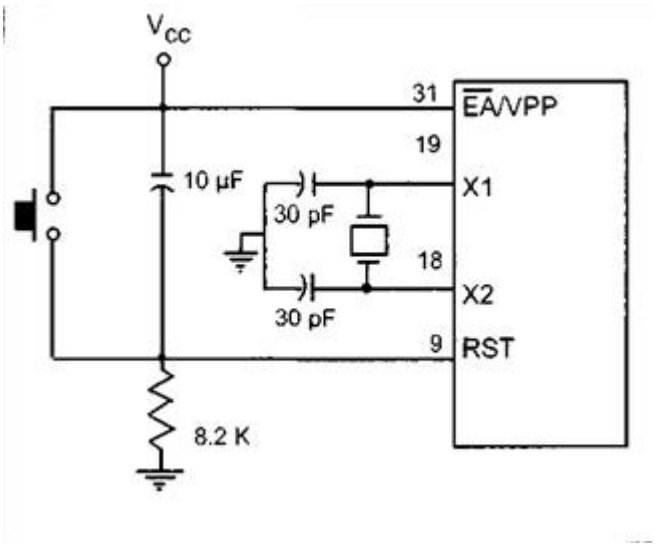


Figure 2-3(b) a momentary switch for reset circuit.

Figure 2-3 (b) uses a momentary switch for reset circuitry.In order for the RESET input to be effective, it must have a minimum duration of two machine cycles. In other words, the high pulse must be high for a minimum of two machine cycles before it is allowed to go low. Here is what the Intel manual says about the Reset circuitry: "When power is turned on, the circuit holds the RST pin high for an amount of time that depends on the capacitor value and the rate at which it charges. To ensure a valid reset the RST pin must be held high long enough to allow the oscillator to start up plus two machine cycles." Although, an 8.2K-ohm resistor and a 10-uF capacitor will take care of the vast majority of the cases, you still need to check the data sheet for the 8051 you are using.

EA-The 8051 family members, such as 8751/52, 89C51/52, or DS89C4xO, all come with on-chip ROM to store programs. In such cases, the EA pin is connected to V_{cc}. For family members such as the 8031 and 8032 in which there is no on-chip ROM. code is stored on an external ROM and is fetched by the 8031/32. Therefore, for the 8031 the EA pin must be connected to GND to indicate that the code is stored externally. EA. which stands for "external access," is pin number 31 in the DIP packages. It is an input pin and must be connected to either V_{cc} or GND. In other words, it cannot be left unconnected.

Ports 0, 1, 2 and 3

As shown in Figure 2-1 the four ports PO, PI, P2, and P3 each use 8 pins, making them 8-bit ports. All the ports upon RESET are configured as input, since PO – P3 have value FFH on them. The following is a summary of features of PO – P3

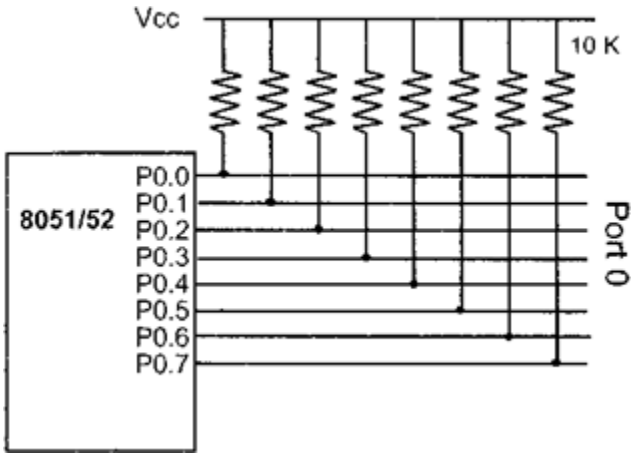


Figure 2-4. Port 0 with Pull-Up Resistors

As shown in Figure 2-1. Port 0 is also designated as ADO – AD7, allowing it to be used for both address and data. When connecting an 8051/31 to an external memory. Port 0 provides both address and data. The 8051 multiplexes address and data through port 0 to save pins. ALE indicates if PO has address or data. When ALE = 0. It provides data DO – D7. But when ALE = 1 it has address A0 –A7". Therefore ALE is used for de-multiplexing address and data with the help of a "4LS373 latch,. In the 8051-based systems where there is no external memory connection, the pins of PO must be connected externally to a 1 OK-ohm pull-up resistor.

Port3

Port 3 occupies a total of 8 pins, pins 10 through 17. It can be used as input or output. P3 does not need any pull-up resistors, the same as PI and P2 did not. Although port 3 is configured as an input port upon reset, this is not the way it is most commonly used. Port 3 has the additional function of providing some extremely important signals such as interrupts. Table 2-1 provides these alternate functions of P3. This information applies to both 8051 and 8031 chips.

.

Table 2-1: Port 3 Alternate Functions

P3Bit	Function	Pin
P3.0	RxD	10
P3.1	TxD	11
P3.2	$\overline{INT0}$	12
P3.3	$\overline{INT1}$	13
P3.4	T0	14
P3.5	T1	15
P3.6	$\overline{WR}$	16
P3.7	$\overline{RD}$	17

Program counter value upon reset

Activating a power-on reset will cause all values in the registers to be lost. Table 2-2 provides a partial list of 8051 registers and their values after power-on reset. From Table 8-2 we note that the value of the PC (program counter) is 0 upon reset, forcing the CPU to fetch the first opcode from ROM memory location 0000. This means that we must place the first byte of opcode in ROM location 0 because that is where the CPU expects to find the first instruction.

Table 2-2: RESET Value of Some 8051 Registers

Register	Reset Value (hex)
PC	0000
DPTR	0000
ACC	00
PSW	00
SP	07
B	00
P0-P3	FF

Machine cycle and crystal frequency

The 8051 uses one or more machine cycles to execute an instruction. The period of machine cycle varies among the different versions of 8051 from 12 clocks in the AT89C51 to 1 clock in the DS89C4xO chip. See Table 2-3. The frequency of the crystal oscillator connected to the X – X2 pins dictates the speed of the clock used in the machine cycle. From Table 2-3, we can conclude that using the same crystal frequency of 12 MHz for both the AT89C51 and DS89C4xO chips gives performance almost 12 times better from the DS89C4xO chip.

Table 2-3: Clocks per Machine Cycle (MC) for Various 8051 Versions

Chip (Maker)	Clocks per Machine Cycle
AT89C51/52 (Atmel)	12
P89C54X2 (Phillips)	6
DS5000 (Dallas Semiconductor)	4
DS89C4x0 (Dallas Semiconductor)	1

Calculations of Count of Timer: Delay generation of 5ms .Assume XTAL is 11.0592 MHz

Algorithm:

A] Main Program:

1. Initialize port as output
2. Set all port pins to logic high
3. Call Delay
4. Clear all Port pins
5. Call Delay
6. Jump to step 2

B] Delay Subroutine:

1. Initialize Timer 0/1 selecting required Mode
2. Load value of TH0 and TL0
3. Clear TF flag
4. Start timer
5. Check TF flag .If it is not set keep checking.
6. Clear TF and stop timer
7. Return back to Main Program

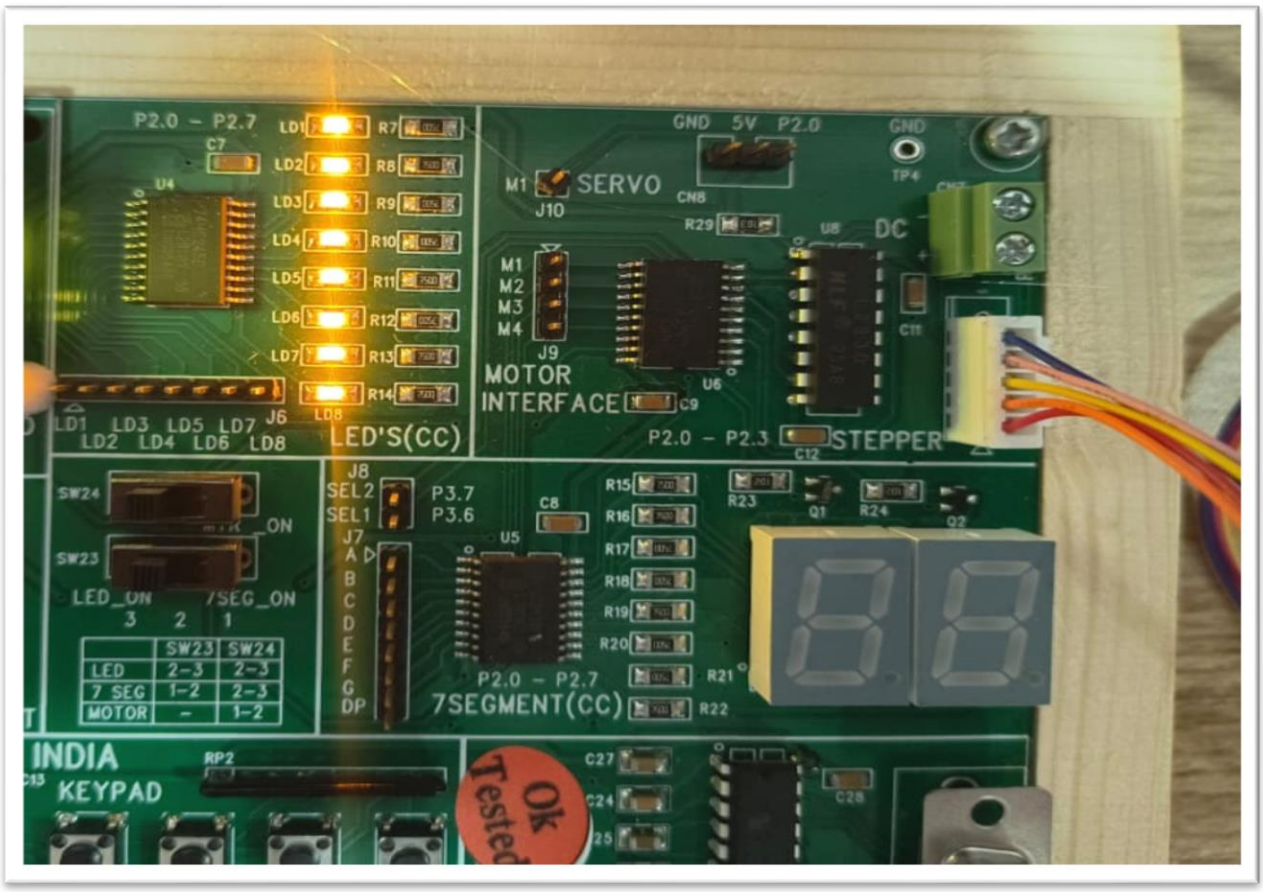
Program :

```
#include <REG51.h>

/*
Function to Add Delay between the Blinking
the function takes argument in nos of microseconds
approximate microsecond time generation
*/
void delaymS(unsigned long x)
{
   while(x > 0 )
   {
      unsigned int i=0;
      for( i=0; i<500;i++);       //approximately 1mS
      x--;
   }
}

void main()
{
   while(1)
   {
      P2 = 0x00;      //Turn off all LEDs
      delaymS(100);   // delay time between the blinking
      P2 = 0xff ;     //Turn on all LEDs
      delaymS(100);   // delay time between the blinking
   }
}
```

Interfacing Output : LED to 8051



Conclusion:

Dr. D. Y. Patil Educational Federation's
Dr. D. Y. Patil Technical Campus
Varale Campus, Pune
Department of Electronics & Computer Engineering

Experiment No. 2

Aim: To interface a Seven Segment Display with the 8051 microcontrollers for displaying numeric digits and counter values using multiplexing technique.

Objective:

1. To study the working of seven-segment displays.
2. To interface seven-segment display with 8051.

Apparatus / Software Required:

1. 8051 / P89V51RD2 microcontroller kit Seven-segment display
2. Keil µVision software (for code development).

Theory :

The 8051's pins are broadly categorized into power and timing, control signals, and I/O ports. **Pins 40 (Vcc) and 20 (GND)** are for power, providing the positive supply voltage and ground, respectively. **Pins 18 (XTAL2) and 19 (XTAL1)** are used to connect an external crystal oscillator, which generates the clock signal for the microcontroller's internal operations.

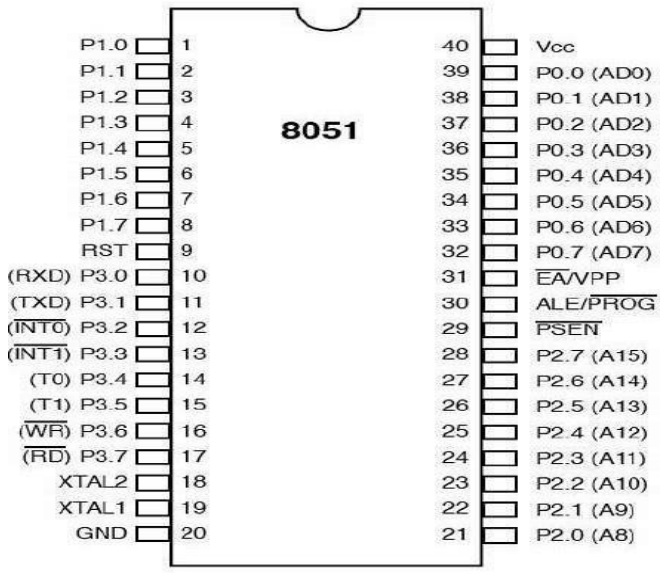


Figure 5.1 Pin Diagram of 8051 Microcontroller

The **control signals** are essential for managing memory and program execution. **Pin 31 (EA/Vpp)** is the External Access Enable pin. When grounded, it forces the 8051 to fetch all instructions from external program memory. When connected to Vcc, it executes instructions from internal program memory until the memory address limit is reached, then automatically switches to external memory. **Pin 30 (ALE/PROG)** stands for Address Latch Enable. It is an output signal that multiplexes the low-order byte of the address from Port 0 during external memory access. **Pin 29 (PSEN)**, or Program Store Enable, is a read strobe signal for external Program memory.

Pins 16 (WR) and 17 (RD) are write and read strobes for external data memory, respectively.

Pin 9 (RST) is the reset pin; a high pulse on this pin resets the microcontroller.

The 8051 microcontroller is an 8-bit microcontroller developed by Intel in 1980 and has since become one of the most widely used microcontrollers in embedded systems. It is part of the MCS-51 family and is known for its simplicity, reliability, and ease of interfacing with peripheral devices. The 8051 is based on an 8-bit CPU, meaning it can process 8 bits of data at a time, and it has a 16-bit address bus that allows it to address up to 64 KB of external memory.

7-segment Display:

A 7-segment display is a form of electronic display device used for displaying decimal numerals. It's a simple, low-cost alternative to more complex displays like dot-matrix or LCD screens. The "seven segments" are actually seven individual LEDs (light-emitting diodes) arranged in a rectangular shape, resembling the number "8." By turning on different combinations of these LEDs, you can form any digit from 0 to 9. Many displays also include a decimal point (DP) as an eighth segment.

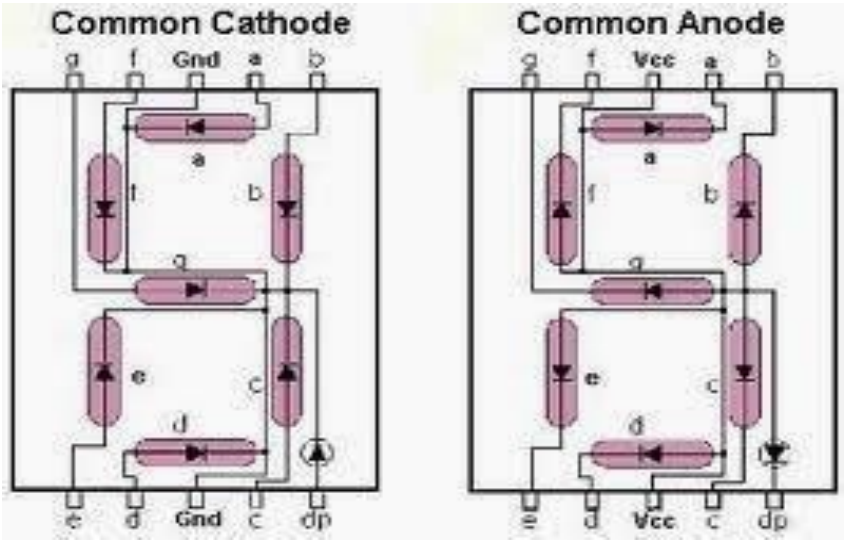


Figure 5.2 Internal Diagram of Seven Segment Display

Working Principle

At its core, a 7-segment display works by the principle of **electroluminescence**, where an LED emits light when an electric current passes through it. The segments are labeled 'a' through 'g' for easy identification. To display a number, you simply provide power to the specific segments that form that number. For example, to display the number "1," you would activate segments 'b' and 'c' while keeping the others off.

A 7-segment display is an electronic display device composed of seven LEDs (segments) arranged to form numerical digits and some characters. The two main types are the common anode and common cathode, and the choice between them depends on the control logic and power supply of the circuit.

A common cathode seven-segment display (CC SSD) is a widely used electronic display device for representing numerical digits and some alphabets. It consists of seven Light Emitting Diodes (LEDs) arranged in a specific pattern to form the shape of the number eight, with each LED segment labeled as

a, b, c, d, e, f, and g. In a common cathode configuration, the cathode (negative terminal) of all seven LEDs is connected together internally and brought out to one or more common pins. This common cathode pin is usually connected to ground (logic 0). To display a number or character, individual segments are illuminated by applying a logic HIGH (positive voltage, typically +5V) to the corresponding anode terminal of each segment. For example, to display the digit “2,” segments a, b, g, e, and d are activated, while the others remain off. The advantage of a common cathode display is that it is simple to interface with microcontrollers like the **8051**, as the microcontroller can directly provide HIGH signals to the desired segment pins to form numbers.

These displays are commonly used in digital clocks, calculators, counters, measuring instruments, and embedded systems due to their low cost, easy availability, and straightforward operation.

Program :

```
#include <REG51.h>

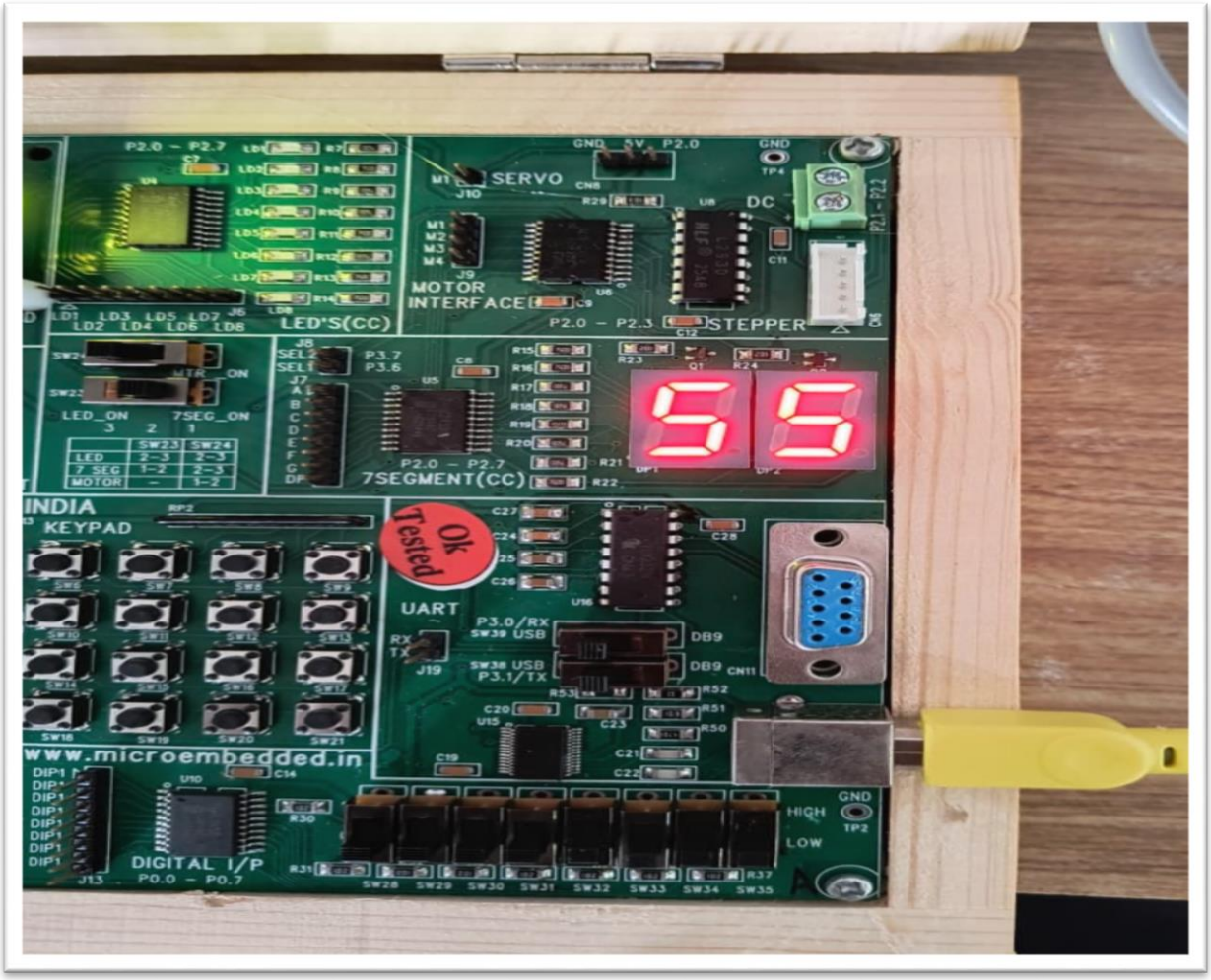
#define SEGPORT  P2    // Port to which 7-SEG is connect
sbit SS1  =P3^6 ;       //segment select 1
sbit SS2  =P3^7 ;       //segment select 2

/*Bit pattern for Dispaying Numerals, starting from 0 to 9*/
const unsigned char LookupTable[]={0x40,0xF9,0x24,0x30,0x19,0x12,0x02,0x78,0x00,0x10};
//common cathode
//const unsigned char
LookupTable[10]={0xF6,0x90,0xE5,0xB5,0x93,0x37,0x77,0x94,0xF7,0xB7}; //common anode

void delaymS(unsigned long x)   // Function to generate a dely of approximate 1 millisecond
{
 unsigned int i,j;
 for(i=0;i<x;i++)
    for(j=0;j<1000;j++);
}

void main()
{
 int i=0;
 unsigned char x=1;
 SEGPORT = 0x00;
 SS1 = 1;
 SS2 = 1;
 while(1)
 {
    for(i=0;i<10;i++)
    {
      SEGPORT = ~LookupTable[i];  // get the pattern from the LookUp Table
      delaymS(100);               // delay for the next number
    }
 }
}
```

Output:



Conclusion:

Dr. D. Y. Patil Educational Federation's
Dr. D. Y. Patil Technical Campus
Varale Campus, Pune
Department of Electronics & Computer Engineering

Experiment No. 3

Aim: To interface a stepper motor with the 8051 microcontrollers for controlling rotational direction, step angle, and speed operation

Objective:

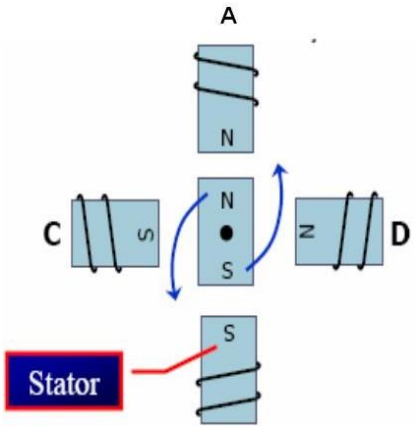
1 To be familiar with interfacing of stepper motor with 8051 microcontroller
2 To be familiar with the calculations of different step angles and speeds of stepper motor

Apparatus: PC keil simulator, 8051 controller kit, Flash magic software , Stepper motor.

Theory:

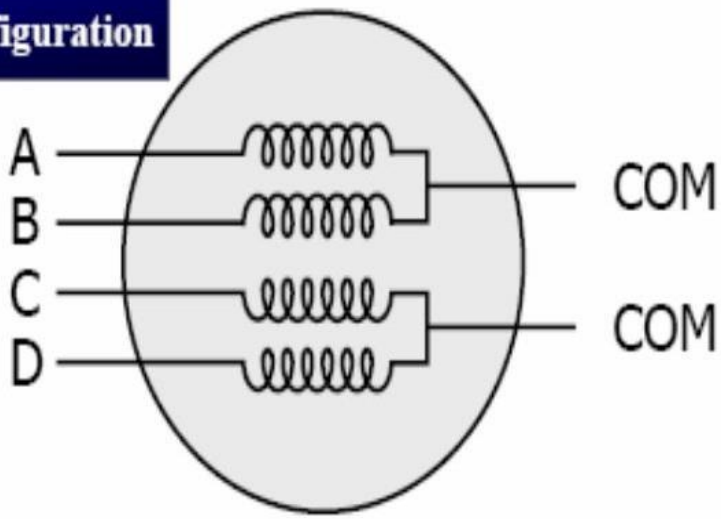
Stepper Motor

• A stepper motor is a widely used device that translates electrical pulses into mechanical movement
• The stepper motor is used for position control in applications such as disk drivers, dot matrix printers, and robotics, etc.
• Every stepper motor has a permanent magnet rotor (also called the shaft) surrounded by a stator



The most common stepper motors have four stator windings that are paired with a center-tapped common.
• This type of stepper motor is commonly referred to as a four-phase stepper motor
• The center tap allows a change of current direction in each of two coils when a winding is grounded, thereby resulting in a polarity change of the stator

Stator Windings Configuration



The stepper motor discussed here has a total of 6 leads, 4 leads representing the four stator windings, 2 commons for the center tapped leads.

• As the sequence of power is applied to each stator winding, the rotor will rotate.

Normal 4-Step Sequence

Clockwise

Step #	Winding A	Winding B	Winding C	Winding D
1	1	0	0	1
2	1	1	0	0
3	0	1	1	0
4	0	0	1	1

Counter-Clockwise

The step angle is the minimum degree of rotation associated with a angles for various motors

• The step per revolution is the total number of steps needed to rotate one complete rotation or 360 degrees. To allow for finer resolutions, all stepper motors allow what is called an 8-step switching sequence.

• It's also called half-stepping, since each step is half of the normal step angle

Half-Step 8-Step Sequence

Clockwise

Step #	Winding A	Winding B	Winding C	Winding D
1	1	0	0	1
2	1	0	0	0
3	1	1	0	0
4	0	1	0	0
5	0	1	1	0
6	0	0	1	0
7	0	0	1	1
8	0	0	0	1

Counter-Clockwise

The relation between RPM (revolutions per minute), steps per revolution, and steps per second is as follow

- Steps/Sec = RPM x Steps per revolution / 60
- After completing every four steps, the rotor moves only one tooth pitch
- The smaller the step angle, the more teeth the motor passes
- ex. In a stepper motor with 200 steps per revolution, its rotor has 50 teeth since 4 x 50 = 200

Algorithm:

1] Start

2] Initialize PORT

3] Send value

4] Call Delay

5] jump to step 3

Program :

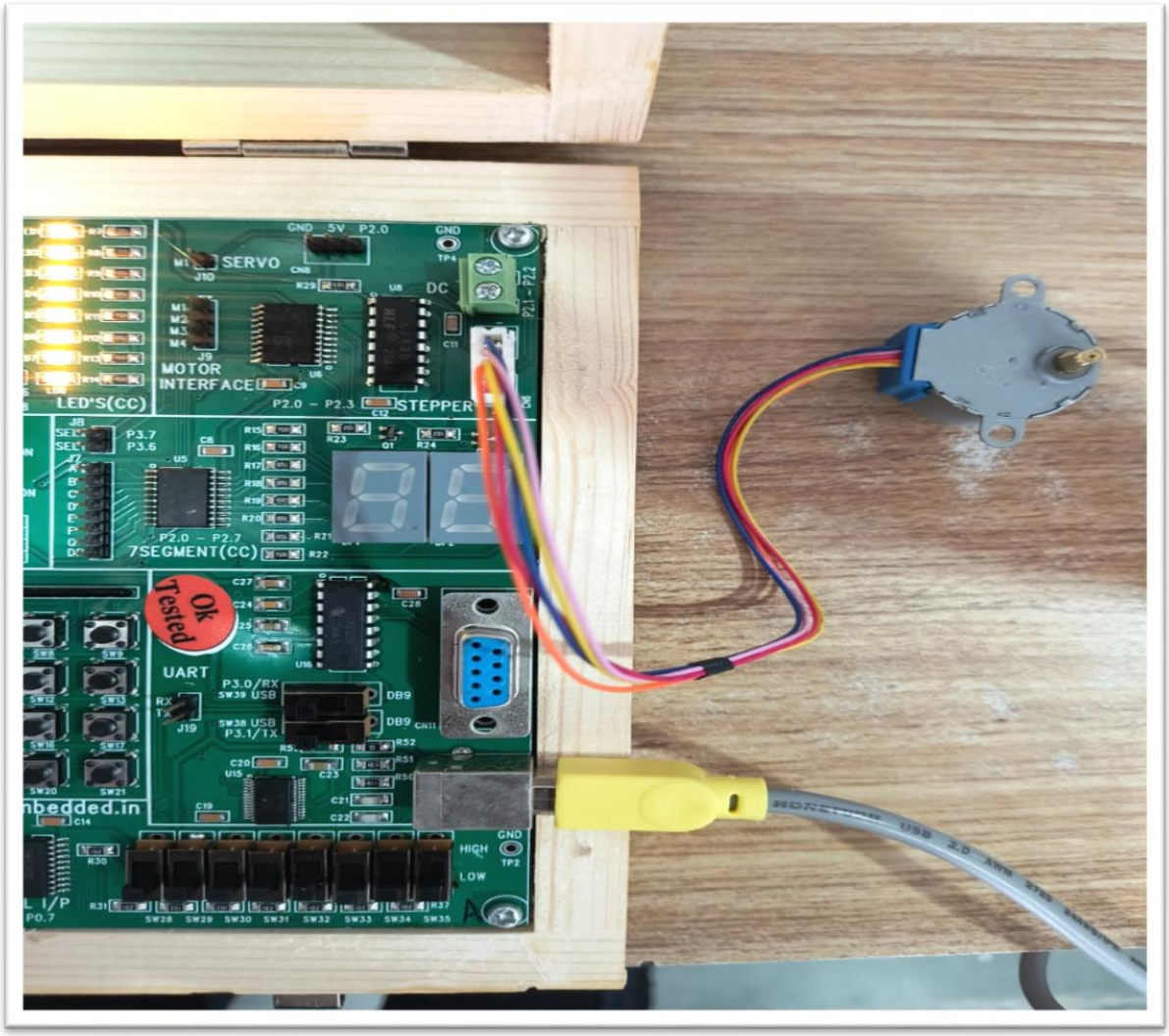
```
#include <REG51.h>

void DelayMs(delay)
{
int i,j;
   for(i=0;i<5;i++)
   {
    for(j=0;j<50;j++);
   }
}

void main(void)
{
   while(1)
   {
         P2 = 0x03;
         DelayMs();
         P2 = 0x06;
         DelayMs();
         P2 = 0x0C;
         DelayMs();
         P2 = 0x09;
         DelayMs();
   }

}
```

Output :



Conclusion:

Dr. D. Y. Patil Educational Federation's
Dr. D. Y. Patil Technical Campus
Varale Campus, Pune
Department of Electronics & Computer Engineering

Experiment No. 4

Title:

To interface a 16×2 LCD display module with the 8051 microcontroller and display alphanumeric characters, messages, and system status information.

Apparatus Required

- PC with Keil simulator software
- 8051 microcontroller kit
- Flash Magic software
- 16×2 LCD display module
- Connecting wires
- Power supply

Theory

A Liquid Crystal Display (LCD) is an electronic display device used to display alphanumeric characters, symbols, numbers, and messages. A 16×2 LCD can display 16 characters per line and has 2 lines, allowing a total of 32 characters to be displayed at a time.

The 16×2 LCD commonly uses the HD44780-compatible controller, which supports both command and data operations.

Working principle

- The 8051 microcontroller sends commands and data to the LCD.
- Commands are used to configure the display, clear the screen, position the cursor, and control the display mode.
- Data represents the characters to be displayed on the LCD.
- The LCD controller receives the instructions and displays the corresponding characters.

LCD pins

The 16×2 LCD has 16 pins with the following functions:

Pin	Name	Function
1	VSS	Ground
2	VDD	+5 V supply
3	VEE/V0	Contrast adjustment
4	RS	Selects command or data register
5	R/W	Selects read or write operation
6	E	Enable signal
7–14	D0–D7	Data bus
15	LED+	Backlight positive
16	LED−	Backlight negative

LCD operating modes

- 8-bit mode: All eight data lines (D0–D7) are used.
- 4-bit mode: Only D4–D7 are used, reducing the number of connections.

In this experiment, the LCD is interfaced in 8-bit mode.

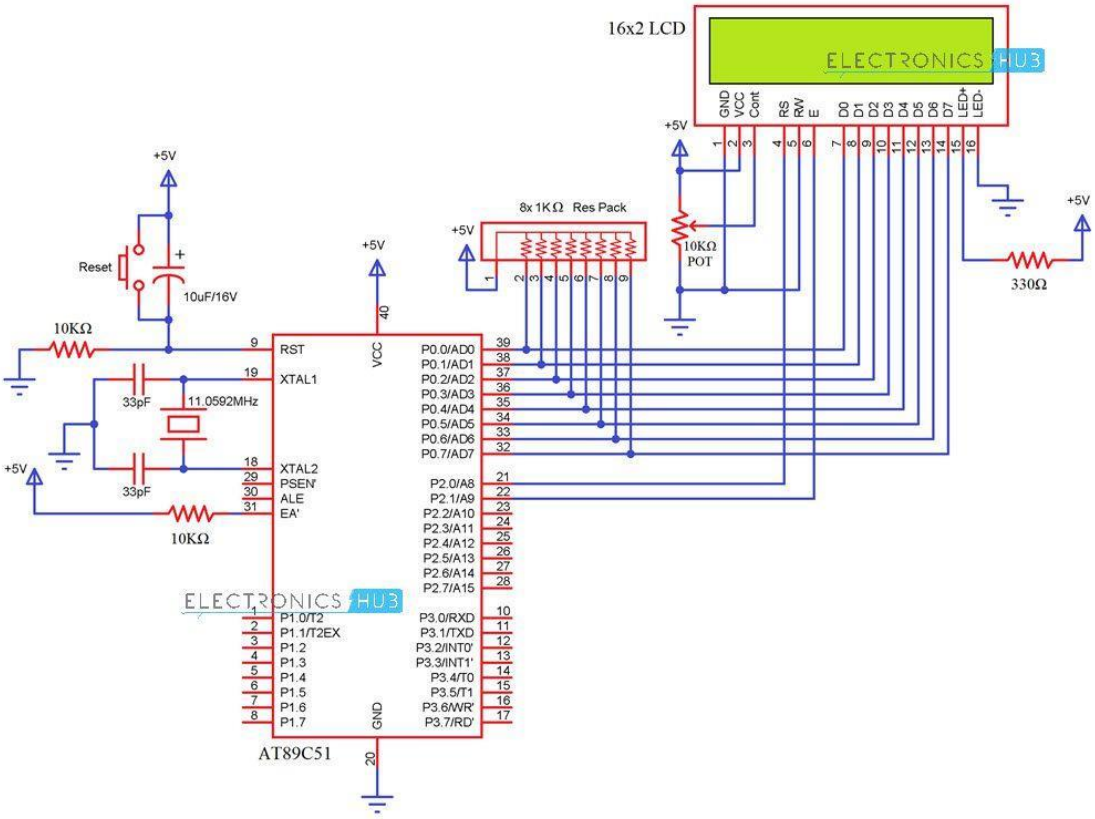
Control signals

- RS = 0: Command register is selected.
- RS = 1: Data register is selected.
- R/W = 0: Data or command is written to the LCD.
- E: A suitable enable pulse allows the LCD to latch the command or data.

Common LCD commands

Command	Function
0x38	8-bit mode, 2-line display
0x0C	Display ON, cursor OFF
0x06	Automatic cursor increment
0x01	Clear display
0x80	Move cursor to first line
0xC0	Move cursor to second line

Interfacing Diagram



Connection description

The following connections are used for the example program:

LCD pin	8051 connection
RS	P3.0
R/W	P3.1
E	P3.2
D0–D7	P2.0–P2.7
VSS	Ground
VDD	+5 V
V0	Contrast potentiometer

The LCD backlight pins should be connected according to the module specifications. Use a current-limiting resistor if required.

Algorithm

1. Start.
2. Initialize the 8051 output ports.
3. Configure the LCD for 8-bit mode and two-line operation.
4. Turn ON the display and set the cursor mode.
5. Clear the LCD screen.
6. Send the cursor-position command for the first line.
7. Send the characters of the required message to the LCD.
8. Send the cursor-position command for the second line.
9. Display the required system status message.
10. Repeat or hold the displayed messages as required.
11. Stop.

Embedded C Program

```
#include <reg51.h>

 #define LCD_DATA P2

 sbit RS = P3^0;
 sbit RW = P3^1;
 sbit EN = P3^2;

 void delay(void)
 {
    unsigned int i, j;

    for (i = 0; i < 100; i++)
        for (j = 0; j < 100; j++);
 }

 void lcd_command(unsigned char cmd)
 {
    LCD_DATA = cmd;
    RS = 0;
    RW = 0;
    EN = 1;
    delay();
    EN = 0;
    delay();
 }
```

```
void lcd_data(unsigned char dat)
{


   LCD_DATA = dat;
   RS = 1;
   RW = 0;
   EN = 1;
   delay();
   EN = 0;
   delay();
}

void lcd_string(char *str)
{
   while (*str)
   {
      lcd_data(*str);
      str++;
   }
}

void main(void)
{
   P2 = 0x00;
   P3 = 0x00;

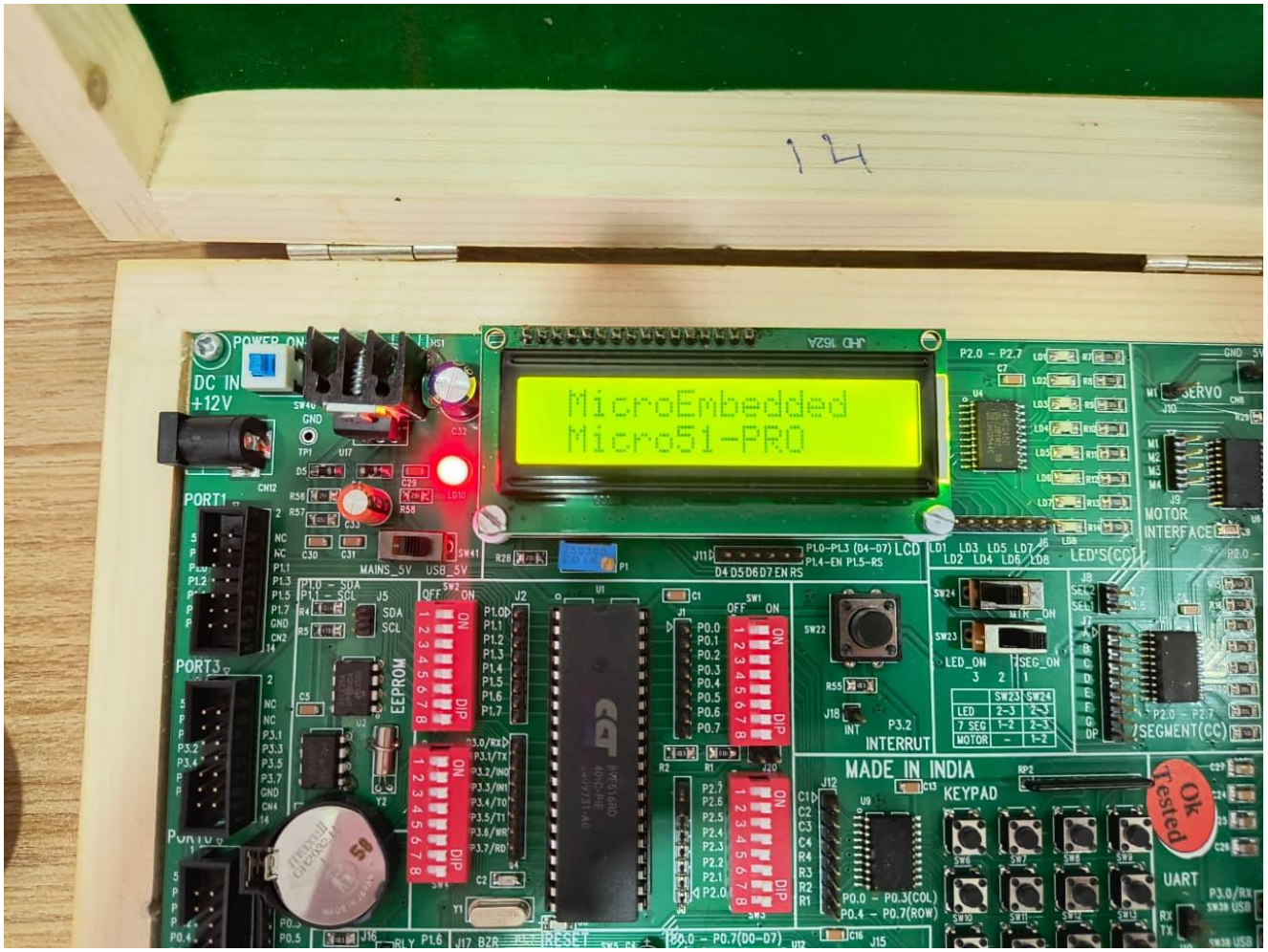
   lcd_command(0x38);
   lcd_command(0x0C);
   lcd_command(0x06);
   lcd_command(0x01);

   lcd_command(0x80);
   lcd_string("MicroEmbedded");

   lcd_command(0xC0);
   lcd_string("Micro51-Pro");

   while (1);
}
```

Output :



Conclusion:

Dr. D. Y. Patil Educational Federation's
Dr. D. Y. Patil Technical Campus
Varale Campus, Pune
Department of Electronics & Computer Engineering

Experiment No. 5

Aim :

To interface an LED with the RL78 microcontroller and write a program for LED blinking to understand digital output port operations and delay generation.

Apparatus Required

- PC with Renesas development software
- RL78 microcontroller development kit
- LED
- Current-limiting resistor (220 Ω to 330 Ω)
- Connecting wires
- USB cable or suitable power supply

Theory

An LED (Light Emitting Diode) is a semiconductor device that emits light when current flows through it in the forward direction. LEDs are widely used as indicators in embedded systems and electronic devices.

The RL78 is a low-power microcontroller family developed by Renesas Electronics. It provides GPIO (General Purpose Input/Output) ports that can be configured to control external devices such as LEDs, switches, and other digital peripherals.

Working Principle

- The RL78 microcontroller configures a GPIO pin as a digital output.
- When the output pin is set HIGH, the LED turns ON in an active-HIGH circuit.
- When the output pin is set LOW, the LED turns OFF.
- By changing the output state repeatedly with a suitable delay, the LED blinks continuously.

Digital Output Port Operation

A GPIO pin can operate as either an input or an output. For LED interfacing, the selected pin is configured as an output.

- Logic HIGH: The output pin produces a high logic level.
- Logic LOW: The output pin produces a low logic level.

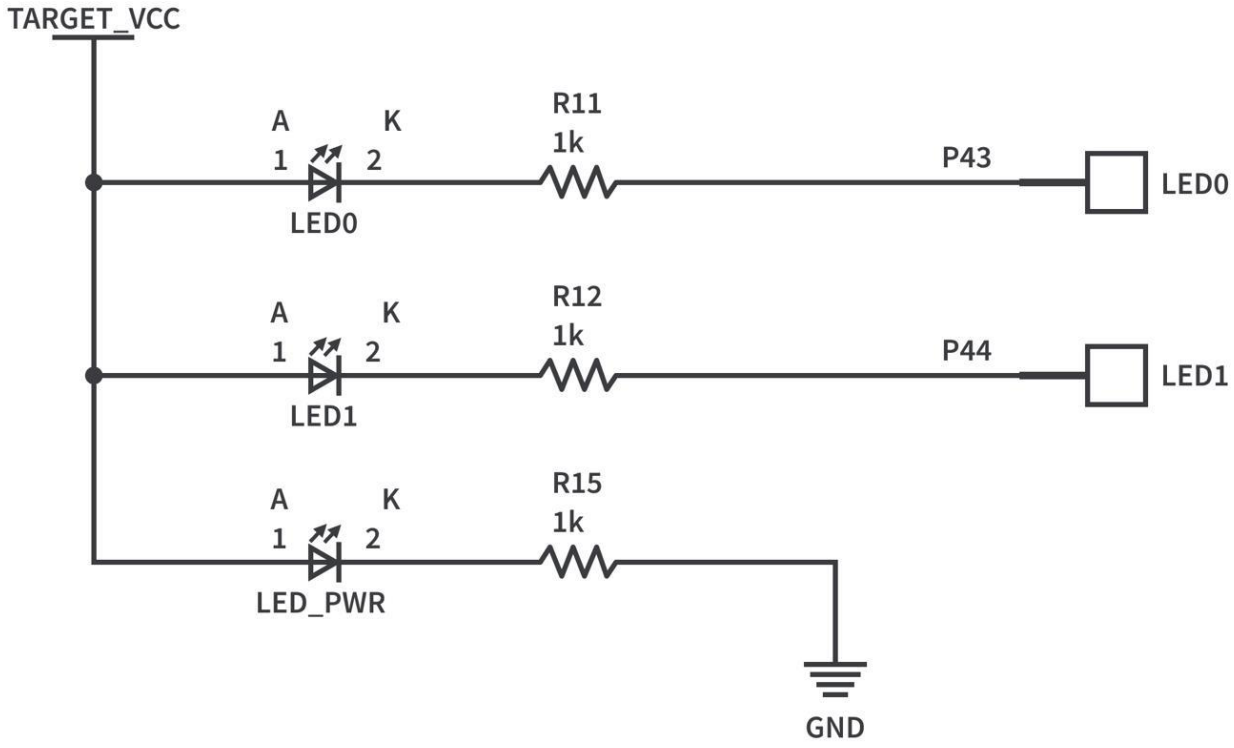
A current-limiting resistor is connected in series with the LED to prevent excessive current from damaging the LED or microcontroller.

Delay Generation

A delay is required between successive ON and OFF states to make the blinking visible. Software delays can be generated using loops, while hardware timers can provide more precise timing.

For example, if the LED remains ON for 500 ms and OFF for 500 ms, the total blinking period is approximately one second.

Interfacing Diagram



Connection Description

- Connect an RL78 GPIO output pin to a current-limiting resistor.
- Connect the resistor to the LED anode.
- Connect the LED cathode to ground.
- Configure the selected GPIO pin as an output.
- Connect the development kit to the appropriate power supply.

The exact port pin and register configuration depend on the RL78 microcontroller model and development board.

Algorithm

1. Start.
2. Initialize the RL78 microcontroller.
3. Configure the selected GPIO pin as an output.
4. Set the output pin HIGH to turn ON the LED.
5. Call the delay routine.
6. Set the output pin LOW to turn OFF the LED.
7. Call the delay routine again.
8. Repeat the process continuously.

Embedded C Program

```
#include "iodefine.h"

void delay(void)
{
   volatile unsigned long i;

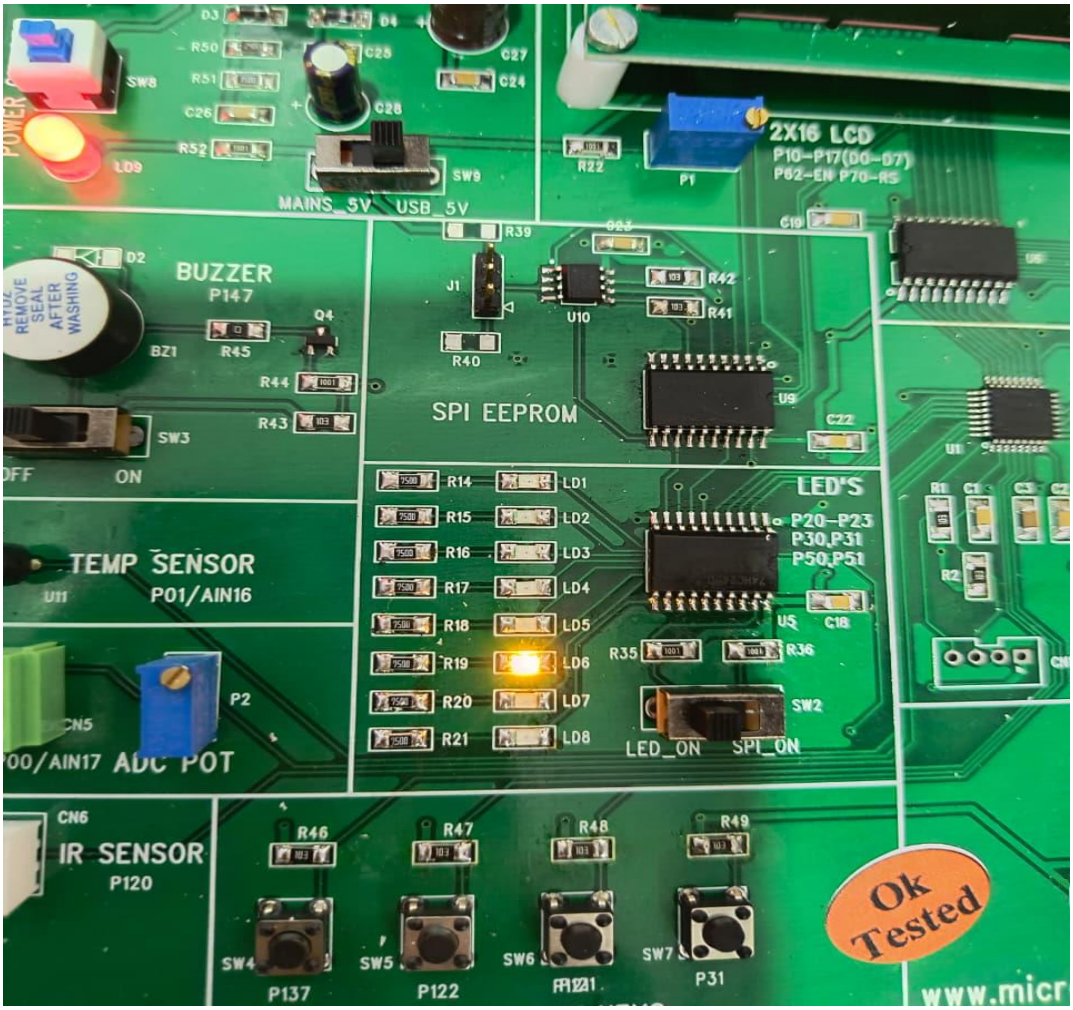
   for (i = 0; i < 50000UL; i++)
   {
      /* Software delay */
   }
}

void main(void)
{
   /* Configure P1.0 as output */
   PM1_bit.no0 = 0;

   while (1)
   {
      P1_bit.no0 = 1;  /* LED ON */
      delay();

      P1_bit.no0 = 0;  /* LED OFF */
      delay();
   }
}
```

Output :



Conclusion:

Dr. D. Y. Patil Educational Federation's
Dr. D. Y. Patil Technical Campus
Varale Campus, Pune
Department of Electronics & Computer Engineering

Experiment No. 6

Aim

To interface a 16×2 LCD display module with the RL78 microcontroller and display alphanumeric characters, messages, and system status information.

Apparatus Required

- PC with Renesas development software
- RL78 microcontroller development kit
- 16×2 LCD display module
- Connecting wires
- Power supply
- USB cable
- Contrast adjustment potentiometer

Theory

A Liquid Crystal Display (LCD) is an electronic display device used to display alphanumeric characters, numbers, symbols, and messages. A 16×2 LCD can display 16 characters per line and has two lines, allowing a total of 32 characters to be displayed at a time.

The RL78 is a low-power microcontroller family developed by Renesas Electronics. It provides GPIO (General Purpose Input/Output) ports that can be configured to interface with external devices such as LCDs, LEDs, switches, and sensors.

Working Principle

- The RL78 microcontroller sends commands and data to the LCD.
- Commands are used to initialize the LCD, clear the display, position the cursor, and control display settings.
- Data represents the characters to be displayed.
- The LCD controller processes the commands and displays the corresponding characters.
- By sending suitable character strings, messages and system status information can be displayed.

LCD Operating Modes

- 8-bit mode: Uses all eight data lines, D0–D7.
- 4-bit mode: Uses four data lines, D4–D7, reducing the number of GPIO connections.

The LCD can operate in either mode, depending on the available GPIO pins and program configuration.

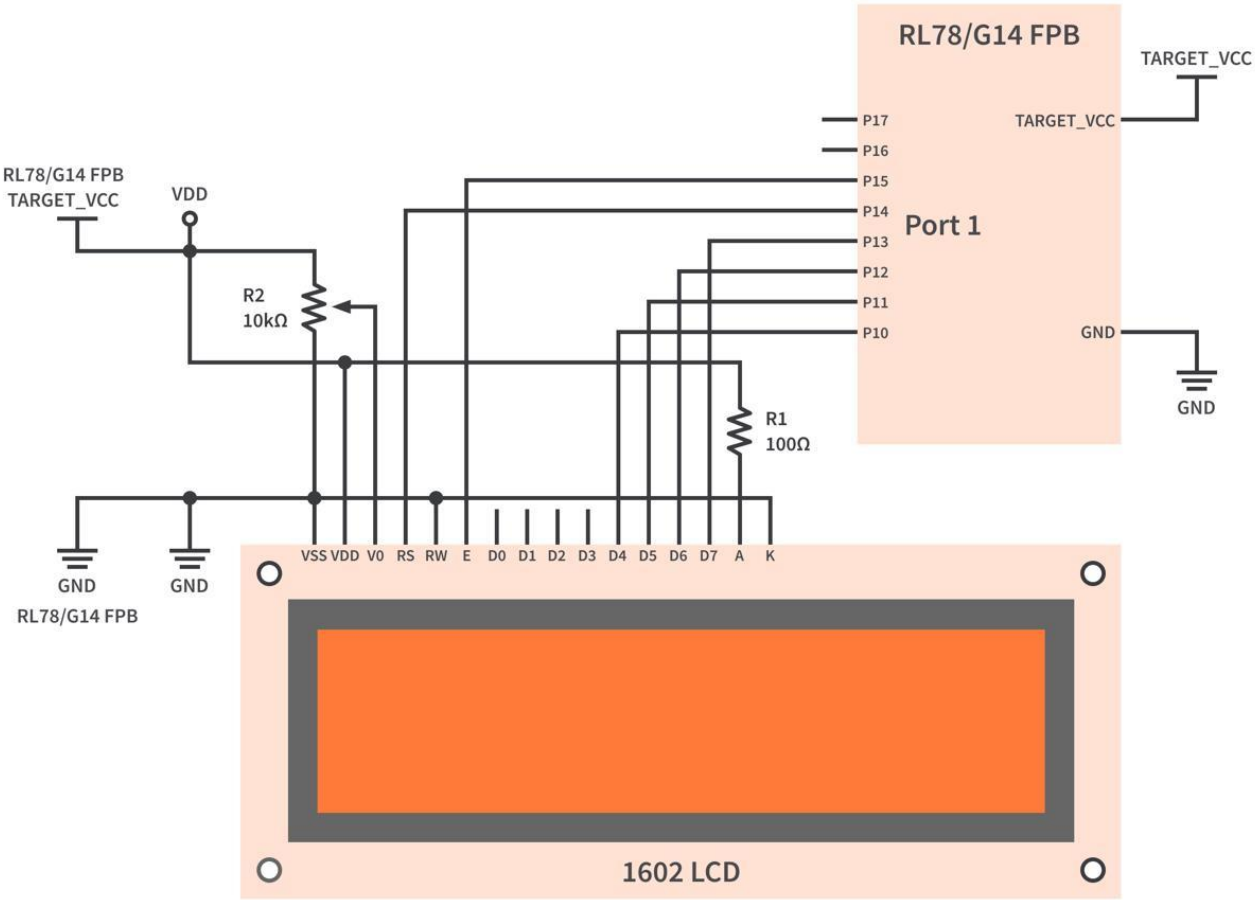
LCD Control Signals

- RS (Register Select): Selects the command register when LOW and the data register when HIGH.
- R/W (Read/Write): Selects the read or write operation. It is usually connected to ground when only writing is required.
- E (Enable): Provides the enable pulse for transferring commands or data.
- D0–D7: Data lines used to transfer commands and character data in 8-bit mode.

Common LCD Commands

Command	Function
0x38	Selects 8-bit mode and two-line display
0x0C	Turns ON display and hides cursor
0x06	Moves cursor forward after writing
0x01	Clears the display
0x80	Positions cursor at the first line
0xC0	Positions cursor at the second line

Interfacing Diagram



Connection Description

For an example 8-bit LCD interface:

LCD Pin	RL78 Connection
RS	Configured GPIO output
R/W	Ground
E	Configured GPIO output
D0–D7	Eight configured GPIO output pins
VSS	Ground
VDD	Supply voltage as specified by the LCD
V0	Contrast adjustment potentiometer
LED+ and LED−	Backlight supply connections

The exact RL78 port pins depend on the development board. Ensure that the LCD voltage levels are compatible with the selected RL78 device.

Algorithm

1. Start.
2. Initialize the RL78 microcontroller.
3. Configure the GPIO pins connected to the LCD as outputs.
4. Initialize the LCD in the selected operating mode.
5. Turn ON the display and configure the cursor.
6. Clear the display.
7. Send the cursor-position command for the first line.
8. Display the required alphanumeric message.
9. Send the cursor-position command for the second line.
10. Display the system status message.
11. Keep the displayed information visible or update it as required.

Embedded C Program

```
#include "iodefine.h"
#define LCD_DATA P2

sbit RS = P1^0;
sbit EN = P1^1;

void delay(void)
{
   volatile unsigned long i;

   for (i = 0; i < 50000UL; i++)
   {
      /* Software delay */
   }
}

void lcd_command(unsigned char cmd)
{
   LCD_DATA = cmd;
   RS = 0;
   EN = 1;
   delay();
   EN = 0;
   delay();
}

void lcd_data(unsigned char dat)
{
   LCD_DATA = dat;
   RS = 1;
   EN = 1;
   delay();
   EN = 0;
   delay();
}

void lcd_string(char *str)
{
   while (*str)
   {
      lcd_data(*str);
```

```
        str++;
    }

}

void main(void)
{
    /* Configure the selected GPIO pins
       as outputs using device-specific settings. */

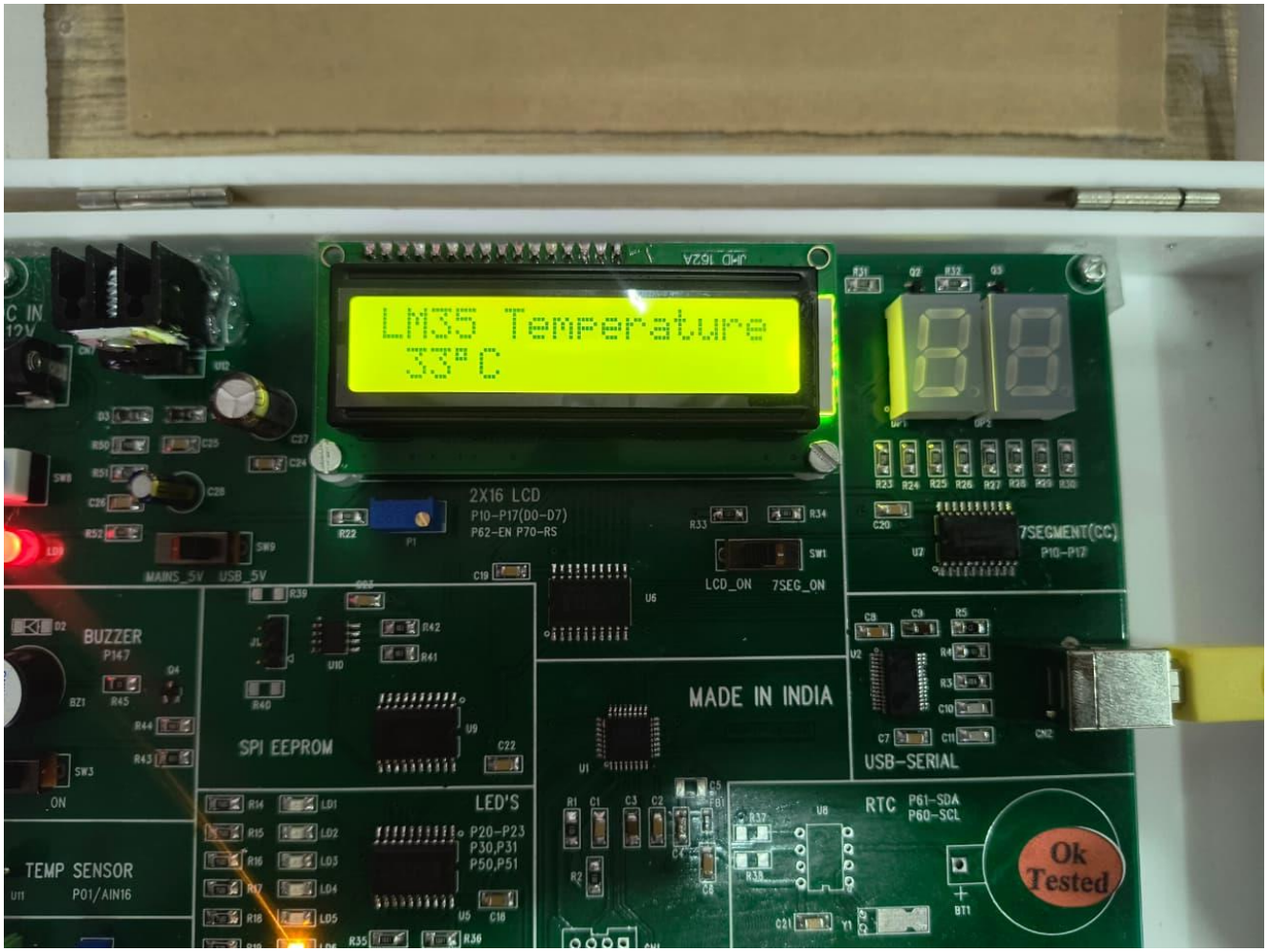
    lcd_command(0x38);
    lcd_command(0x0C);
    lcd_command(0x06);
    lcd_command(0x01);

    lcd_command(0x80);
    lcd_string("LM35 Temperature");

    lcd_command(0xC0);
    lcd_string("33 C");

    while (1);
}
```

Output :



Conclusion: